

Babu Banarasi Das University



Case Study ON

E-commerce Transaction Analysis using Hadoop

Submitted to:

Mr. Vikash (IBM)

Submitted by:

Group G

Member's name:-

- Swarnim vishwakarma
- Sumit gangwar
- Tamanna kumari
- Tashvi sumit

PROJECT

E-commerce Transaction Analysis using Hadoop MapReduce

► Project Description:

This project focuses on analysing e-commerce transaction data using the Hadoop ecosystem. The main objective is to process large-scale data efficiently and extract useful insights such as total revenue and top-selling products. Traditional systems fail to handle such large datasets effectively, so this project uses distributed computing for better performance and scalability.

► Architecture:

- The architecture of this project follows a standard Big Data pipeline using Hadoop ecosystem components.

► Data Flow:

Raw Dataset → HDFS → MapReduce Processing → Processed Data → Hive Analysis → Insights

► Explanation:

- Raw dataset (e-commerce data) is first stored in HDFS
- MapReduce is used to process the data and calculate total revenue per product
- Processed data is stored back in HDFS
- Hive is used to perform analytical queries
- Final insights are generated from the results

► Tools Used:

- Hadoop Distributed File System (HDFS)
- MapReduce (Hadoop Streaming using Python)
- Hive (for data analysis)
- Cloudera
- ChatGPT (Generative AI tool)

► **Technologies:**

- Python (for mapper and reducer)
- CSV dataset
- Hadoop Ecosystem

► **Use of Generative AI in the Project:**

- Generative AI tools were used to assist in development and understanding of Big Data concepts.

► **Areas Where AI Was Used:**

1. Code Generation

- Used to generate mapper and reducer programs

2. Debugging

- Errors were identified and fixed using AI

3. Query Writing

- Hive queries were generated and optimized

4. Concept Understanding

- Helped in understanding:
 - HDFS architecture
 - MapReduce workflow
 - Hive operations

► **Example Prompt Used:**

Write a Python MapReduce program to calculate total revenue per product.

► **Benefits of Using AI:**

- Faster development
- Reduced coding errors
- Better understanding
- Improved efficiency

► **Validation:**

- All AI-generated code was manually tested and verified

► **Dataset Description:**

- The dataset used is an e-commerce transaction dataset containing product sales and purchase details.

► **Dataset Structure:**

- OrderID – Unique order ID
- Product – Product name
- Category – Product category
- Price – Price of product
- Quantity – Quantity purchased
- Date – Purchase date

► **Dataset Source:**

- Custom dataset created for project demonstration

► **Purpose of Dataset:**

- Analyze product sales
- Calculate revenue
- Identify top-selling products
- Generate insights

► Sample Data:

```
</> csv

OrderID,Product,Category,Price,Quantity,Date
101,Mobile,Electronics,15000,1,2024-01-10
102,Shoes,Fashion,2000,2,2024-01-10
103,Laptop,Electronics,50000,1,2024-01-11
104,Headphones,Electronics,3000,3,2024-01-11
105,T-shirt,Fashion,800,4,2024-01-12
106,Watch,Accessories,2500,2,2024-01-12
107,Tablet,Electronics,20000,1,2024-01-13
108,Jeans,Fashion,1500,2,2024-01-13
109,Camera,Electronics,30000,1,2024-01-14
110,Sandals,Fashion,1200,3,2024-01-14
111,Speaker,Electronics,4000,2,2024-01-15
112,Backpack,Accessories,1800,2,2024-01-15
113,Smartwatch,Electronics,7000,1,2024-01-16
114,Jacket,Fashion,2500,2,2024-01-16
115,Printer,Electronics,10000,1,2024-01-17
116,Sunglasses,Accessories,1500,3,2024-01-17
117,Keyboard,Electronics,1200,2,2024-01-18
118,Mouse,Electronics,800,3,2024-01-18
119,Dress,Fashion,2200,2,2024-01-19
120,PowerBank,Electronics,1500,3,2024-01-19
```

► Conclusion:

- In this project, a complete Big Data pipeline was successfully implemented using Hadoop ecosystem components.
- The dataset was stored in HDFS, processed using MapReduce to calculate total revenue per product, and analyzed using Hive to extract insights such as top-selling products.
- The project helped in understanding distributed data processing.
- Generative AI assisted in code generation, debugging, and concept understanding.

◆ PROBLEM STATEMENT 1: E-commerce Analysis using Hadoop Streaming

► Aim:

To analyze e-commerce dataset and calculate total revenue using MapReduce.

Step-01: Create CSV file

cat > ecommerce.csv

```
[cloudera@quickstart ~]$ cd project
[cloudera@quickstart project]$ cat>bd.txt
OrderID,Product,Category,Price,Quantity,Date
101,mobile,electronics,15000,1,2024-01-10
102,shoes,fashion,2000,2,2024-01-10
103,laptop,electronics,50000,1,2024-01-11
104,headphones,electronics,3000,3,2024-01-11
105,T-shirt,fashion,800,4,2024-01-12
106,watch,accessories,2500,2,2024-01-12
107,tablet,electronics,20000,1,2024-01-13
108,jeans,fashion,1500,2,2024-01-13
109,camera,electronics,30000,1,2024-01-14
110,sandals,fashion,1200,3,2024-01-14
111,speaker,electronics,4000,2,2024-01-15
112,backpack,accessories,1800,2,2024-01-15
113,smartwatch,electronics,7000,1,2024-01-16
114,jacket,fashion,2500,2,2024-01-16
115,printer,electronics,10000,1,2024-01-17
116,sunglasses,accessories,1500,3,2024-01-17
117,keyboard,electronics,1200,2,2024-01-18
118,mouse,electronics,800,3,2024-01-19
120,powerbank,electronics,1500,3,2024-01-19
```

Step-02: Create HDFS directory

hdfs dfs -mkdir /user/cloudera/ecommerce

```
[cloudera@quickstart project]$ hdfs dfs -mkdir /user/cloudera/swarnimvish
[cloudera@quickstart project]$ hdfs dfs -mkdir /user/cloudera/swarnimvish/ecommerce
```

Step-03: Upload dataset

hdfs dfs -put ecommerce.csv /user/cloudera/ecommerce

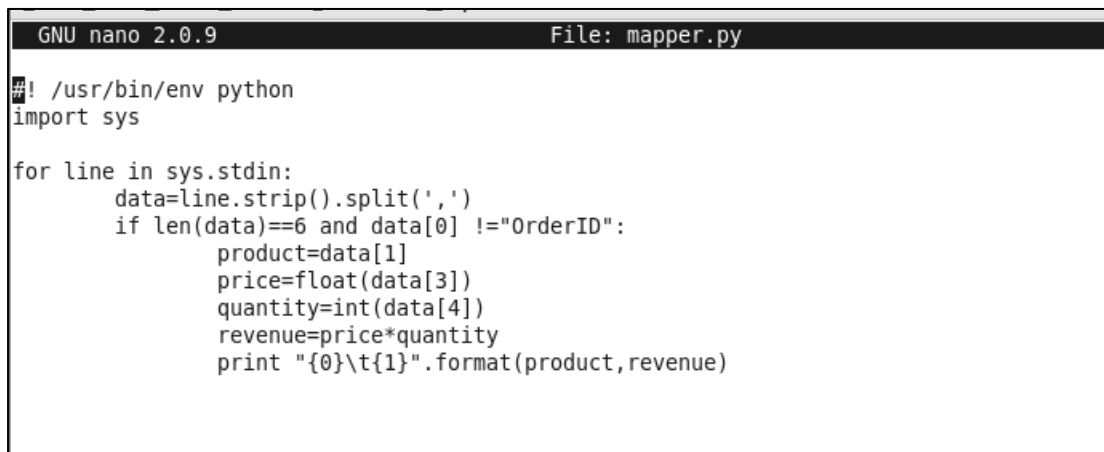
hdfs dfs -ls /user/cloudera/ecommerce

```
[cloudera@quickstart project]$ hdfs dfs -put bd.csv /user/cloudera/swarnimvish/ecommerce
[cloudera@quickstart project]$ hdfs dfs -ls /user/cloudera/swarnimvish
Found 1 items
drwxr-xr-x  - cloudera cloudera      0 2026-04-23 09:33 /user/cloudera/swarnimvish/ecommerce
[cloudera@quickstart project]$
```

Step-04: Create mapper.py

```
#!/usr/bin/env python
import sys

for line in sys.stdin:
    data = line.strip().split(',')
    if len(data) == 6 and data[0] != "OrderID":
        product = data[1]
        price = float(data[3])
        quantity = int(data[4])
        revenue = price * quantity
        print "{0}\t{1}".format(product, revenue)
```

A screenshot of a terminal window with a dark background. The title bar at the top shows "GNU nano 2.0.9" on the left and "File: mapper.py" on the right. The terminal displays the same Python code as the previous block, with a cursor at the end of the last line.

```
GNU nano 2.0.9 File: mapper.py
#!/usr/bin/env python
import sys

for line in sys.stdin:
    data=line.strip().split(',')
    if len(data)==6 and data[0] != "OrderID":
        product=data[1]
        price=float(data[3])
        quantity=int(data[4])
        revenue=price*quantity
        print "{0}\t{1}".format(product, revenue)
```

Step-05: Create reducer.py

```
#!/usr/bin/env python
import sys
current_product = None
total = 0
for line in sys.stdin:
    Line=line.strip()
    product, revenue = line.strip().split('\t')
    revenue = float(revenue)
    if current_product == product:
        total += revenue
    else:
        if current_product:
            print "{0}\t{1}".format(current_product,total)
        current_product = product
        total = revenue
```

```
if current_product:
    print "{0}\t{1}".format(current_product,total)
```

```
GNU nano 2.0.9                               File: reducer.py
#!/usr/bin/env/python
import sys

current_product=None
total=0

for line in sys.stdin:
    line=line.strip()
    product,revenue=line.strip().split("\t")
    revenue=float(revenue)

    if current_product==product:
        total=total+revenue
    else:
        if current_product:
            print "{0}\t{1}".format(current_product,total)
            current_product=product
            total=revenue
if current_product:
    print "{0}\t{1}".format(current_product,total)
```

Step-06: Make executable

chmod +x mapper.py reducer.py

```
cloudera@quickstart project]$
[cloudera@quickstart project]$ chmod +x mapper.py reducer.py
[cloudera@quickstart project]$ ls -l
total 16
-rw-rw-r-- 1 cloudera cloudera 826 Apr 23 09:25 bd.csv
-rw-rw-r-- 1 cloudera cloudera 826 Apr 23 09:25 bd.txt~
-rwxrwxr-x 1 cloudera cloudera 253 Apr 23 09:51 mapper.py
-rwxrwxr-x 1 cloudera cloudera 375 Apr 23 10:05 reducer.py
[cloudera@quickstart project]$
```


- Local test

cat ecommerce.csv|python mapper.py

```
[cloudera@quickstart project]$ cat bd.csv |python mapper.py
mobile 15000.0
shoes 4000.0
laptop 50000.0
headphones 9000.0
T-shirt 3200.0
watch 5000.0
tablet 20000.0
jeans 3000.0
camera 30000.0
sandals 3600.0
speaker 8000.0
backpack 3600.0
smartwatch 7000.0
jacket 5000.0
printer 10000.0
sunglasses 4500.0
keyboard 2400.0
mouse 2400.0
powerbank 4500.0
```

cat ecommerce.csv|python mapper.py |sort|python reducer.py

```
[cloudera@quickstart project]$ cat bd.csv |python mapper.py|sort|python reducer.py
backpack 3600.0
camera 30000.0
headphones 9000.0
jacket 5000.0
jeans 3000.0
keyboard 2400.0
laptop 50000.0
mobile 15000.0
mouse 2400.0
powerbank 4500.0
printer 10000.0
sandals 3600.0
shoes 4000.0
smartwatch 7000.0
speaker 8000.0
sunglasses 4500.0
tablet 20000.0
T-shirt 3200.0
watch 5000.0
[cloudera@quickstart project]$ █
```

Step-07: Run job

```
hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar \  
-files mapper.py,reducer.py \  
-input /user/cloudera/ecommerce/ecommerce.csv \  
-output /user/cloudera/ecommerce/output \  
-mapper "python mapper.py" \  
-reducer "python reducer.py"
```

```
[cloudera@quickstart project]$ hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar \  
> -files mapper.py,reducer.py \  
> -input /user/cloudera/swarnimvish/ecommerce/bd.csv \  
> -output /user/cloudera/ecommerce/output \  
> -mapper "python mapper.py" \  
> -reducer "python reducer.py" \  
packageJobJar: [] [/usr/jars/hadoop-streaming-2.6.0-cdh5.4.2.jar] /tmp/streamjob8186400762315128107.jar tmpDir=null  
26/04/23 10:50:28 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032  
26/04/23 10:50:29 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032  
26/04/23 10:50:30 INFO mapred.FileInputFormat: Total input paths to process : 1  
26/04/23 10:50:30 INFO mapreduce.JobSubmitter: number of splits:2  
26/04/23 10:50:31 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1776957947287_0001  
26/04/23 10:50:31 INFO impl.YarnClientImpl: Submitted application application_1776957947287_0001  
26/04/23 10:50:32 INFO mapreduce.Job: The url to track the job: http://quickstart.cloudera:8088/proxy/application_177  
6957947287_0001/  
26/04/23 10:50:32 INFO mapreduce.Job: Running job: job_1776957947287_0001  
26/04/23 10:50:46 INFO mapreduce.Job: Job job_1776957947287_0001 running in uber mode : false  
26/04/23 10:50:46 INFO mapreduce.Job: map 0% reduce 0%  
26/04/23 10:51:34 INFO mapreduce.Job: map 100% reduce 0%  
26/04/23 10:51:45 INFO mapreduce.Job: map 100% reduce 100%  
26/04/23 10:51:46 INFO mapreduce.Job: Job job_1776957947287_0001 completed successfully  
26/04/23 10:51:47 INFO mapreduce.Job: Counters: 49  
    File System Counters  
        FILE: Number of bytes read=334  
        FILE: Number of bytes written=342208  
        FILE: Number of read operations=0  
        FILE: Number of large read operations=0  
        FILE: Number of write operations=0  
        HDFS: Number of bytes read=1491  
        HDFS: Number of bytes written=290  
        HDFS: Number of read operations=9  
        HDFS: Number of large read operations=0  
        HDFS: Number of write operations=2  
    Job Counters  
        Launched map tasks=2  
        Launched reduce tasks=1  
        Data-local map tasks=2  
        Total time spent by all maps in occupied slots (ms)=92433  
        Total time spent by all reduces in occupied slots (ms)=8782  
        Total time spent by all map tasks (ms)=92433  
        Total time spent by all reduce tasks (ms)=8782  
        Total vcore-seconds taken by all map tasks=92433  
        Total vcore-seconds taken by all reduce tasks=8782  
        Total megabyte-seconds taken by all map tasks=94651392
```

Step-08: View output

hdfs dfs -cat /user/cloudera/ecommerce/output/part-00000

```
[cloudera@quickstart project]$ hdfs dfs -cat /user/cloudera/ecommerce/output/part-00000
T-shirt 3200.0
backpack      3600.0
camera 30000.0
headphones    9000.0
jacket  5000.0
jeans   3000.0
keyboard    2400.0
laptop  50000.0
mobile  15000.0
mouse   2400.0
powerbank   4500.0
printer 10000.0
sandals  3600.0
shoes    4000.0
smartwatch 7000.0
speaker  8000.0
sunglasses 4500.0
tablet  20000.0
watch   5000.0
[cloudera@quickstart project]$
```

outcome:

Learned how to process large datasets in a distributed way by converting raw data into key-value pairs using Mapper and aggregating results using Reducer to calculate total revenue per product.

Also understood data transformation, parallel processing, and how Hadoop Streaming executes Python code on big data.

◆ PROBLEM STATEMENT 2: Hive Analysis

► Aim:

To analyze processed data using Hive.

Step-01: Open Hive

hive

```
[cloudera@quickstart project]$ hive

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive> █
```

Step-02: Create table

```
CREATE TABLE ecommerce_rev (
product STRING,
revenue FLOAT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY '\t';
```

```
hive>
>
> create table ecommerce_rev(
> product string,
> revenue float)
> row format delimited
> fields terminated by '\t';
OK
Time taken: 3.567 seconds
```

Step-03: Load data

```
LOAD DATA INPATH '/user/cloudera/ecommerce/output'
INTO TABLE ecommerce_rev;
```

```
hive> load data inpath '/user/cloudera/ecommerce/output'
> into table ecommerce_rev;
Loading data to table default.ecommerce_rev
chgrp: changing ownership of 'hdfs://quickstart.cloudera:8020/user/hive/warehouse/ecommerce_rev/part-00000': User doe
s not belong to hive
Table default.ecommerce_rev stats: [numFiles=1, totalSize=290]
OK
Time taken: 1.14 seconds
```

Step-04: View data

SELECT * FROM ecommerce_rev;

```
hive> select * from ecommerce_rev;
OK
T-shirt 3200.0
backpack      3600.0
camera 30000.0
headphones    9000.0
jacket  5000.0
jeans   3000.0
keyboard    2400.0
laptop  50000.0
mobile  15000.0
mouse    2400.0
powerbank   4500.0
printer 10000.0
sandals  3600.0
shoes    4000.0
smartwatch  7000.0
speaker  8000.0
sunglasses  4500.0
tablet  20000.0
watch    5000.0
Time taken: 0.799 seconds, Fetched: 19 row(s)
```

Step-05: Top products

SELECT * FROM ecommerce_rev ORDER BY revenue DESC;

```
hive> select * from ecommerce_rev order by revenue desc;
Query ID = cloudera_20260423110808_8bfeb338-5e00-4315-a929-0e36a38f8d79
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1776957947287_0003, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1776957947287_0003/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1776957947287_0003
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-04-23 11:08:25,930 Stage-1 map = 0%, reduce = 0%
2026-04-23 11:08:34,550 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 2.0 sec
2026-04-23 11:08:45,300 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 3.86 sec
MapReduce Total cumulative CPU time: 3 seconds 860 msec
Ended Job = job_1776957947287_0003
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 3.86 sec HDFS Read: 5946 HDFS Write: 290 SUCCESS
Total MapReduce CPU Time Spent: 3 seconds 860 msec
OK
laptop 50000.0
camera 30000.0
tablet 20000.0
mobile 15000.0
printer 10000.0
headphones 9000.0
speaker 8000.0
smartwatch 7000.0
watch 5000.0
jacket 5000.0
powerbank 4500.0
sunglasses 4500.0
shoes 4000.0
sandals 3600.0
backpack 3600.0
T-shirt 3200.0
jeans 3000.0
keyboard 2400.0
mouse 2400.0
Time taken: 30.664 seconds, Fetched: 19 row(s)
hive> cloudera
```

Step-06: Top 5 products

SELECT * FROM ecommerce_rev ORDER BY revenue DESC LIMIT 5;

```
hive> select * from ecommerce_rev order by revenue desc limit 5;
Query ID = cloudera_20260423111010_4e937cb3-170e-4992-8512-c4f0547a8cfd
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1776957947287_0004, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1776957947287_0004/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1776957947287_0004
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-04-23 11:10:20,737 Stage-1 map = 0%, reduce = 0%
2026-04-23 11:10:30,710 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 2.14 sec
2026-04-23 11:10:41,471 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 4.06 sec
MapReduce Total cumulative CPU time: 4 seconds 60 msec
Ended Job = job_1776957947287_0004
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 4.06 sec HDFS Read: 6070 HDFS Write: 76 SUCCESS
Total MapReduce CPU Time Spent: 4 seconds 60 msec
OK
laptop 50000.0
camera 30000.0
tablet 20000.0
mobile 15000.0
printer 10000.0
Time taken: 31.604 seconds, Fetched: 5 row(s)
hive> █
```

Outcome:

Learned how to analyze processed data using SQL-like queries to sort, filter, and identify top-selling products and revenue trends.

Also understood how Hive simplifies complex data analysis, supports structured querying on big data, and helps generate meaningful business insights for decision-making.